

A* navigation

In this task, you will complete a part of a "follow me" project for the Turtlebot robot. You will complete a path planning node using the A* algorithm, a path tracking node using a PID controller and a goal pose publisher node from and AprilTag detection. First, you will implement these algorithms in the simulated environment (except the goal pose publisher), and subsequently deploy them onto the physical robot.

Point Distribution

The points for this part of the task are distributed as follows:

- **10 points** for completing the A* algorithm
- **10 points** for creating and tuning the PID regulator
- **8 points** for creating a publisher for the goal pose from an AprilTag detection

The deadline for this assignment is **April 19th**. Submissions must be completed **in person**, ideally during the final lab session. If you need to submit your work at a different time, please email me in advance to arrange an alternative date. Penalty is 1,5 points/day of delay.

Task responsible: Libor Zelenka (zelenli1@fel.cvut.cz^[1])

The Turtlebot Platform

The Turtlebot is powered by Robot Operating System (ROS) [ROS](#)^[2], a middleware that handles communication between sensors, actuators, and your code. We deploy the stack using Aptainer to ensure a consistent, portable environment. Start by installing your environment using the instructions mentioned here: [Setting up ROS](#)^[3] (Linux guide, full support); see: [Alternative \(Unsupported\) Installation Options](#)^[4] if you can't use Linux. To understand how ROS works and how to use it effectively, please read through this page: [Using ROS](#)^[5]

Once you have developed your algorithms, you can deploy and test them on the physical Turtlebot. For working on the real robot, please refer to this guide: [TurtleBot Lab Guide](#)^[6].

Before working with the physical robot, all students must complete the mandatory training, which will take place during the second lab session.

Even though you will be working with ROS, the main goal of this task is to teach you more about algorithms and their implementation. Don't worry if you don't fully understand ROS yet. The provided code will handle the ROS-specific details for you.

Simulation Launch

If you have a Linux laptop, here is how to set up the environment and launch the task:

```
git clone https://gitlab.fel.cvut.cz/robofab/deploy.git
cd deploy/scripts/install_apptainer
./download_apptainer_image
```

```
deploy/scripts/start_apptainer_aro
# console prompt should start with ARO now
```

The start_apptainer_aro script not only starts the Apptainer container, but also creates the appropriate ROS workspace, clones the student packages to the workspace, builds the workspace, and sources the workspace. Details of this process are described [here](#)[7].

After starting the Apptainer container, try to run the simulation which will be part of your first homework using the command:

```
# in 1st terminal window:
ros2 run rmw_zenoh_cpp rmw_zenohd
# keep zenohd running and call this in the 2nd terminal:
ros2 launch astar_nav astar_sim.launch.xml
```

Real Robot Deployment

Deploying your code to the physical Turtlebot requires a few more steps and specific network configurations. Instead of duplicating those instructions here, please refer directly to the guide here: [TurtleBot Lab Guide](#)[8]

A* path planning

A* is a popular graph search algorithm used for finding the shortest, obstacle-free path. For its understanding, please refer to [A* Wiki](#)[9], [SciPy implementation](#)[10] or to these videos



A* (A Star) Search Algorithm - Comput
Computerphile



Watch on



A* Pathfinding (E01: algorithm explana
Sebastian Lague



Watch on

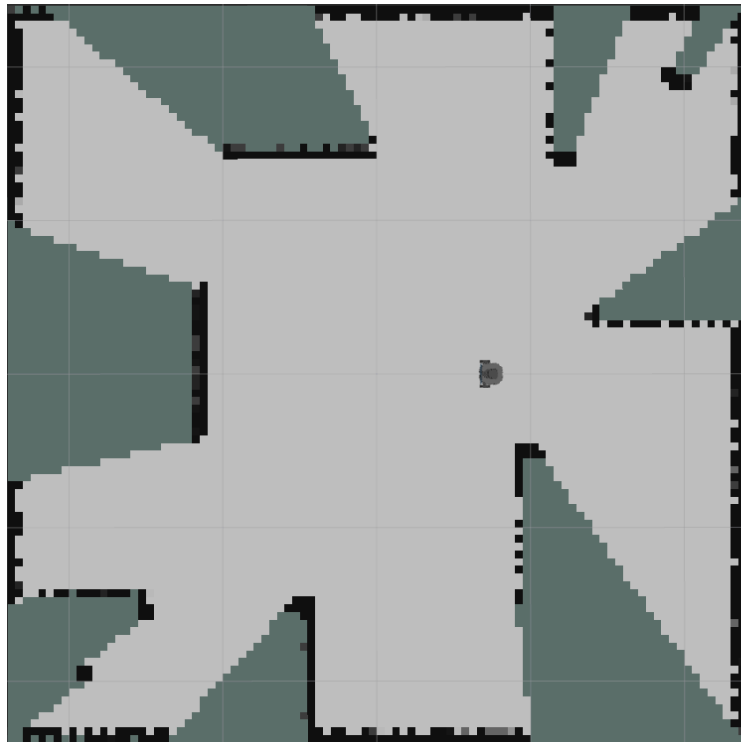
Your task will be to implement the algorithm using a 2D occupancy grid. This will require modifying the grid to avoid collisions with obstacles.

Occupancy Grid

Occupancy is a common representation used for path planning. The grid divides the world into 2D or 3D cells, which can be **occupied** (= object/not traversable), **free** (=no objects/traversable) or **unknown**. The grid on the Turtlebot robot is constructed from a LiDAR scan and creates a 2D map, which is already given to you in an array format.

To work with *grid_map*, you can access its data using row and column as follows *grid_map[row, col]*. The data stored inside the grid map represents each cell's state. Which are:

- * Free: represented with a value of **0**
- * Occupied: represented with a value of **100**
- * Unknown: represented with a value of **-1**

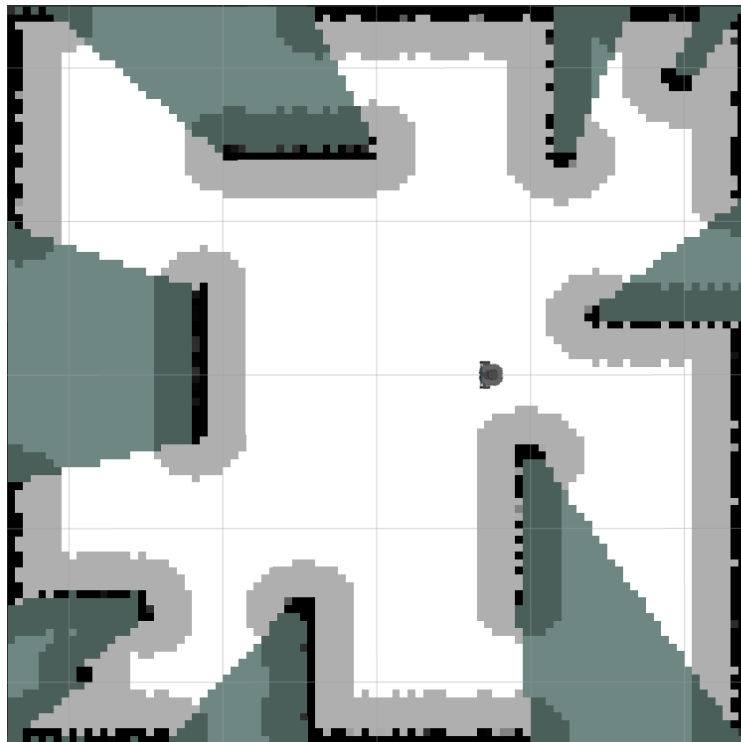


[11]

Avoiding obstacles

Planning a path in a plain occupancy grid would result in the shortest path, but it might not be physically feasible for our robot. In order to account for the robot's size, we construct a new grid where we inflate the obstacles so the robot doesn't get too close to them. The grid map you are working with is already inflated, so there is no need to modify it.

The inflation process is straightforward. For each occupied cell in the original grid map, a circle is drawn with a radius slightly larger than the robot's radius. You can see the result here:



[12]

Goal setting in sim

To set the goal for A* planning we use the `\goal_pose` topic for the real robot we will use AprilTags but more about that later. In RViz the publish to this topic can be triggered with a **2D Goal Pose** button which is located in the toolbar.

PID path tracking

Once A* has generated a collision-free path, we need a way to follow it. A Proportional-Integral-Derivative (PID) regulator is a widely used control loop feedback mechanism that continuously calculates an error value and applies a correction to minimise it.

Your task is to implement a complete PID regulator; however, use your judgment to determine whether to utilize all three components or tune it to operate as a simpler P, PI, or PD controller. To get a feel for how these regulators work, please visit [Learn About PID\[13\]](#) or [PID drone control\[14\]](#). For more information about control theory and the PID regulator's place in it, please visit [The PID Controller & Theory Explained\[15\]](#) or [PID Control\[16\]](#).

Defining the Setpoint and Error

In control theory, a system always works to reach a desired target condition, known as the **setpoint**. For your Turtlebot, the setpoint is the ideal state of being perfectly aligned with the path generated by A*.

The **error** is the measurable difference between that ideal setpoint and the robot's actual state. To follow the planned trajectory, the Turtlebot must continuously compute this deviation. In path tracking, this typically involves measuring two main types of error:

- **Cross-track error:** How far the robot is situated from the path line.
- **Heading error:** The difference between the robot's current orientation and the desired direction.

Generating Motor Commands and Reaching Steady State

You will use the calculated **heading error** to generate your motor commands, effectively telling the robot how to correct itself and return to the setpoint.

First, use your regulator to translate the heading error into an angular velocity command to steer the robot. Next, use the magnitude of this error to determine the linear speed. You need to dynamically calculate the forward speed so that if the heading error is large, the robot slows down to complete the turn smoothly. Conversely, if the error is small and the robot is facing the right direction, speed up.

Finally, by adjusting your chosen P, I, and D parameters, you will tune the regulator to help the robot reach a steady state—a condition where it smoothly and consistently follows the path. Proper tuning ensures the robot navigates without excessive oscillation (overcorrecting past the setpoint) and helps eliminate **steady-state error** (a constant, lingering offset from the path, which is typically corrected by the Integral term).

There is a vast amount of complex control theory and mathematical formulas dedicated to calculating the “perfect” PID parameters or controllers in general. However, for navigating a Turtlebot, a systematic *trial-and-error* approach is usually more than enough.

System Integration: The "Follow Me" Behaviour

The overarching objective of this exercise is to implement a “Follow Me” behaviour, functioning as the primary integration point for the path planning (A*) and trajectory tracking (PID) modules. In this context, “following” entails dynamically updating the robot's target coordinates based on real-time environmental observations.

To achieve this, the method for setting the dynamic target will depend on your testing environment. In the simulator, you will update the target coordinates using a standard 2D goal pose in RViz. When transitioning to the physical robot, the target will be defined by a designated visual marker (AprilTag).

AprilTag Goal Pose Publisher

The computer vision pipeline required to detect the AprilTag and compute its real-world pose is provided. This system continuously publishes the tag's spatial information into the ROS **TF (transform) tree**.

Your objective is to implement a ROS node that acts as an interface between the vision system and the A* planner. To facilitate this, a hollow node structure containing a timer and its corresponding callback function will be provided. The implementation consists of two primary components:

Transform Acquisition

The TF tree maintains the relationship between multiple coordinate frames over time. As the vision node automatically integrates the AprilTag frame into the TF tree, direct image processing is not required on your end.

Within the provided timer callback, you will utilize the `tf2_ros` library—specifically employing `tf2_ros::TransformListener` and `tf2_ros::Buffer`—to query the transform between the robot's reference frame and the AprilTag frame. The objective is to retrieve the most recent translation vector (x, y, z) of the tag relative to the robot.

Goal Pose Publication

Upon successfully retrieving the transform data, the node must parse and publish this information in a format compatible with the A* planner.

You will instantiate a publisher designated to output a `geometry_msgs/msg/PoseStamped` message on the `/goal_pose` topic. When populating this message, ensure the following parameters are correctly assigned:

- The `header` must contain the appropriate `frame_id` (corresponding to the reference frame of the acquired transform) and the current timestamp.
- The translation coordinates derived from the TF transform must be mapped to the respective `pose.position` fields.

Continuous execution of this node ensures the Turtlebot dynamically updates its A* target to mirror the physical location of the AprilTag, thereby achieving the intended "Follow Me" behaviour.

Detailed implementation aspects, identification of exact coordinate frames, and methodologies for analyzing the TF tree (e.g., utilizing tools such as `rqt_tf_tree`) will be discussed comprehensively in the laboratories on 30.3.

Hypertext Links

- [1] <mailto:mailto:zelenli1@fel.cvut.cz?subject=STAI>
- [2] <https://www.ros.org/>
- [3] <https://cw.fel.cvut.cz/wiki/courses/aro/tutorials/ros>
- [4] https://cw.fel.cvut.cz/wiki/courses/aro/tutorials/alternative_install
- [5] <https://cw.fel.cvut.cz/wiki/courses/aro/tutorials/lab01>
- [6] <https://cw.fel.cvut.cz/wiki/courses/aro/tutorials/turtlebots>
- [7] https://cw.fel.cvut.cz/wiki/courses/aro/tutorials/ros#behind_the_scenes
- [8] <https://cw.fel.cvut.cz/wiki/courses/aro/tutorials/turtlebots>
- [9] https://en.wikipedia.org/wiki/A*_search_algorithm
- [10] https://www.w3schools.com/python/scipy/scipy_graphs.php
- [11] https://cw.fel.cvut.cz/wiki/_detail/courses/becm36stai/tutorials/follow_me/plainocupancymap.png?id=courses%3Abecm36stai%3Atutorials%3Afollow_me%3Astart
- [12] https://cw.fel.cvut.cz/wiki/_detail/courses/becm36stai/tutorials/follow_me/infatedoccupancymap.png?id=courses%3Abecm36stai%3Atutorials%3Afollow_me%3Astart
- [13] <https://learnaboutpid.com/>
- [14] <https://pid-drone-control.vercel.app/>
- [15] <https://www.ni.com/en/shop/labview/pid-theory-explained.html>
- [16] <https://thomasfermi.github.io/Algorithms-for-Automated-Driving/Control/PID.html>

[courses/becm36stai/tutorials/follow_me/start.txt](#) · Last modified: 2026/03/31 19:32 by zelenli1

reports and suggestions [ServiceDesk CTU](#)